MNR University

[Established under the Telangana State Private Universities

(Establishment and Regulation) (Amendment) Act No. 11 of 2024]

MNR Nagar, Fasalwadi, Sangareddy District, Gr. Hyderabad – 502294

MNRU School of Engineering & Technology

and

Artificial Intelligence

A.Y. : 2025-26

B.Tech I Year - II Semester

25CSE104 - Software Crafting & Sculpting Lab

Name of the Student: N. Madwika Lakshmi

Program : B.Tech (Hons.) CSE (AI&ML)

Section : B

Hall Ticket Number : 25RBAIM0064

Index

STOCK MAINTENANCE SYSTEM

TASK 1: DEVELOPMENT OF PROBLEM STATEMENT

Title

Stock Maintenance System

Introduction

Stock Maintenance System is a software application used to manage inventory in an organization, shop, warehouse, or company. Maintaining stock manually using registers and spreadsheets often leads to errors, data redundancy, stock mismanagement, and delays in report generation.

The proposed Stock Maintenance System automates inventory management activities such as stock entry, stock updates, supplier management, purchase records, sales records, and report generation.

Problem Statement

Many organizations still use manual methods for inventory management. These methods face several challenges:

- Difficulty in tracking stock availability.
- Human errors in stock calculations.
- Time-consuming report generation.
- Duplicate records.
- Inaccurate inventory updates.
- Poor supplier management.
- Difficulty in identifying low-stock items.

To overcome these issues, a computerized Stock Maintenance System is required.

Objectives

Primary Objectives

1. Automate stock management.
2. Reduce human errors.
3. Improve inventory tracking.
4. Generate accurate reports.

Secondary Objectives

1. Maintain supplier details.
2. Manage sales and purchases.
3. Monitor stock levels.
4. Generate alerts for low stock.

Scope of the System

Included Features

- Product Management
- Supplier Management
- Purchase Management
- Sales Management
- Inventory Tracking
- Report Generation

Excluded Features

- Online Payment Integration
- Mobile Application
- AI Prediction Module

Benefits

- Improved efficiency
- Faster report generation
- Reduced paperwork
- Better decision making
- Accurate inventory management

Conclusion

The Stock Maintenance System provides an efficient solution for managing inventory. It reduces manual work, improves accuracy, and enhances organizational productivity.

TASK 2: SOFTWARE REQUIREMENT SPECIFICATION (SRS)

1. Introduction

Purpose

The purpose of the Stock Maintenance System is to automate inventory management and provide efficient stock control.

Scope

The system manages products, suppliers, purchases, sales, and reports.

2. Functional Requirements

Product Module

- Add Product
- Update Product
- Delete Product
- View Product

Supplier Module

- Add Supplier
- Update Supplier
- Delete Supplier

Purchase Module

- Record Purchase
- Update Stock Quantity

Sales Module

- Record Sales
- Reduce Stock Quantity

Report Module

- Generate Daily Reports
- Generate Monthly Reports

3. Non-Functional Requirements

Performance

- Response time < 3 seconds

Security

- Login Authentication
- Password Encryption

Reliability

- System Availability 99%

Usability

- User-friendly Interface

4. Hardware Requirements

- Processor: Intel i3 or above
- RAM: 4GB
- Hard Disk: 100GB

5. Software Requirements

- Windows/Linux
- MySQL
- Java/Python

6. Constraints

- Internet required for cloud deployment.
- Authorized users only.

Conclusion

The SRS document clearly defines system functionality and quality requirements.

TASK 3: SOFTWARE CONFIGURATION MANAGEMENT (SCM) AND RISK MANAGEMENT

Software Configuration Management

Definition

SCM controls software changes throughout development.

Objectives

- Version Control
- Change Tracking
- Configuration Identification

SCM Activities

Configuration Identification

Identify modules:

- Login Module
- Product Module
- Supplier Module
- Report Module

Version Control

Git Repository Management

Change Management

Review and approve changes.

Release Management

Maintain stable software releases.

Risk Management

Risk Identification

Risk	Impact
Hardware Failure	High

Data Loss	High
Security Breach	High
Requirement Change	Medium
Team Absence	Medium

Risk Analysis

Technical Risks

- Software Bugs
- Database Failure

Project Risks

- Schedule Delays
- Resource Shortage

Business Risks

- Customer Requirement Changes

Risk Mitigation

- Regular Backups
- Security Updates
- Frequent Testing
- Team Training

Conclusion

SCM and Risk Management ensure controlled software development and reduced project risks.

Task 4: Study and Usage of Design Phase CASE Tool

1. Introduction

Visual Paradigm is a CASE (Computer-Aided Software Engineering) tool used for software design and UML modeling. It helps developers create diagrams, visualize system architecture, and improve software documentation.

For the Stock Maintenance System, Visual Paradigm was used to design UML diagrams such as Use Case and Class Diagrams.

2. Purpose of Using Visual Paradigm

- Analyze system requirements.
- Create UML diagrams.
- Visualize system components and relationships.
- Improve documentation.
- Support communication among developers.

3. System Requirements

- Operating System: Windows/Linux/macOS
- RAM: 4 GB minimum
- Storage: 1 GB free space

4. Installation Procedure

1. Visit the Visual Paradigm website.
2. Download the Community Edition.
3. Install the software.
4. Launch and create a new project.

5. Creating a New Project

1. Open Visual Paradigm.
2. Select Create New Project.
3. Enter Stock Maintenance System.
4. Click OK.

6. Main Components

- Diagram Toolbar: Create UML diagrams.

- Model Explorer: View project structure.
- Drawing Area: Workspace for diagrams.
- Properties Panel: Edit diagram elements.

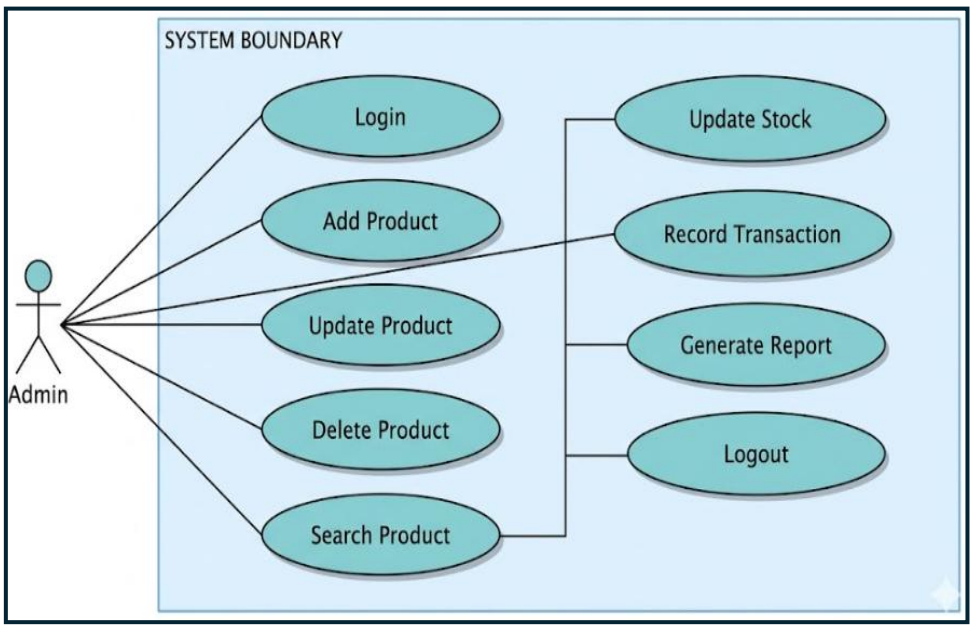
7. UML Diagrams Used

7.1 Use Case Diagram

Actor: Admin

Use Cases:

- Login
- Add Product
- Update Product
- Delete Product
- Search Product
- Update Stock
- Record Transaction
- Generate Report
- Logout



Procedure:

1. Create Use Case Diagram.
2. Add Admin actor.
3. Add use cases.
4. Connect actor with use cases.
5. Save diagram.

7.2 Class Diagram

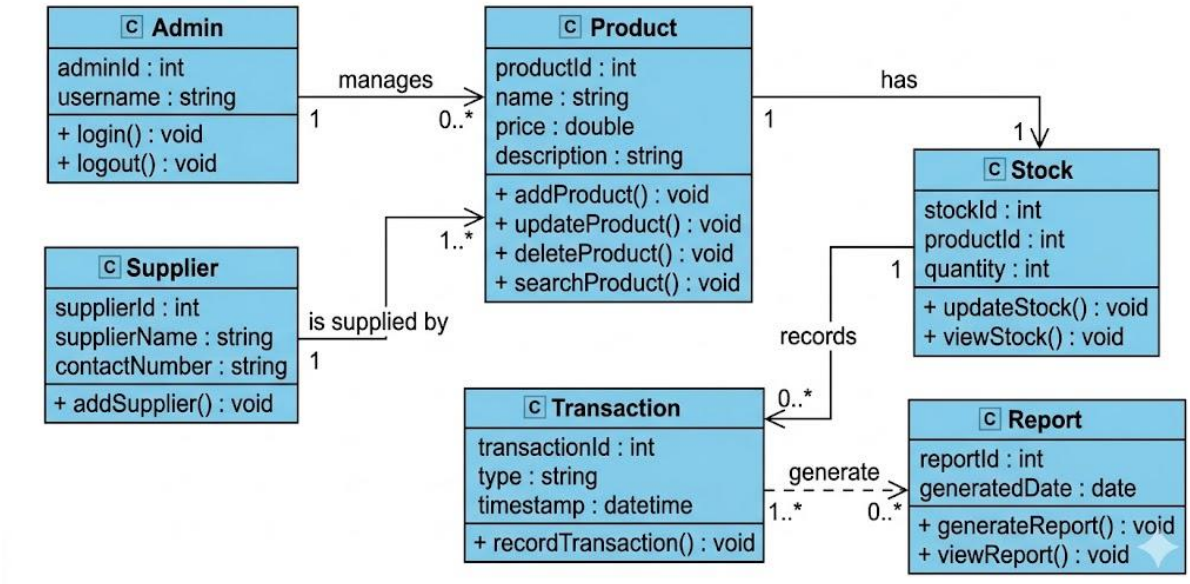
Class	Main Functions
Admin	login(), logout()
Product	addProduct(), updateProduct(), deleteProduct(), searchProduct()
Supplier	addSupplier()

Stock	updateStock(), viewStock()
Transaction	recordTransaction()
Report	generateReport(), viewReport()

Relationships:

- Admin manages Product.
- Product is supplied by Supplier.
- Product has Stock.
- Stock records Transactions.
- Transactions generate Reports.

7.2 Class Diagram: Product, Stock, and Transaction Management System



8. Benefits of Visual Paradigm

- Easy drag-and-drop modeling.
- Supports UML diagrams.
- Improves documentation.

9. Conclusion

Visual Paradigm was successfully used to design the Stock Maintenance System. The Use Case and Class Diagrams provided a clear understanding of system functionality and structure, improving documentation and development planning.

TASK – 5: Preparation and Development of ER Diagram

Aim

To develop an Entity Relationship (ER) Diagram for the Stock Maintenance System using a CASE tool such as Visual Paradigm.

Objective

The ER Diagram represents the entities, attributes, and relationships in the Stock Maintenance System. It helps design the database structure and manage inventory data efficiently.

Entities and Attributes

Admin

- Admin_ID (PK)
- Username
- Password
- Email

Supplier

- Supplier_ID (PK)
- Supplier_Name
- Contact_No
- Address

Product

- Product_ID (PK)
- Product_Name
- Category
- Price
- Quantity
- Supplier_ID (FK)

Stock

- Stock_ID (PK)
- Product_ID (FK)

- Quantity_Available
- Last_Updated

Transaction

- Transaction_ID (PK)
- Product_ID (FK)
- Transaction_Type
- Quantity
- Transaction_Date

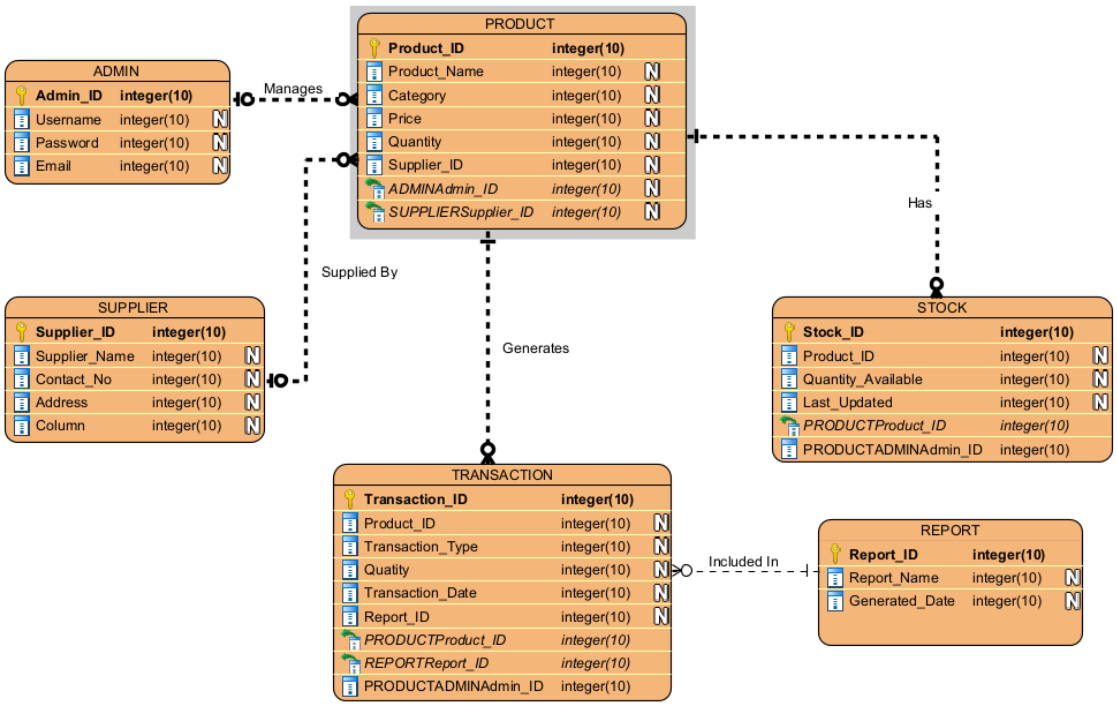
Report

- Report_ID (PK)
- Report_Name
- Generated_Date

Relationships

Relationship	**Cardinality**
Admin manages Product	1 : M
Supplier supplies Product	1 : M
Product has Stock	1 : 1
Product generates Transaction	1 : M
Transaction included in Report	M : 1

ER Diagram



Conclusion

The ER Diagram provides a clear representation of the Stock Maintenance System database. It defines the relationships between Admin, Supplier, Product, Stock, Transaction, and Report entities, supporting efficient inventory management and report generation.

TASK – 6: Preparation and Development of Data Flow Diagrams (DFD)

Aim

To develop Context Diagram, Level 0 DFD, and Level 1 DFD for the Stock Maintenance System using Visual Paradigm.

Objective

To represent the flow of data between users, processes, and databases in the Stock Maintenance System.

1. Context Diagram

External Entity

Admin

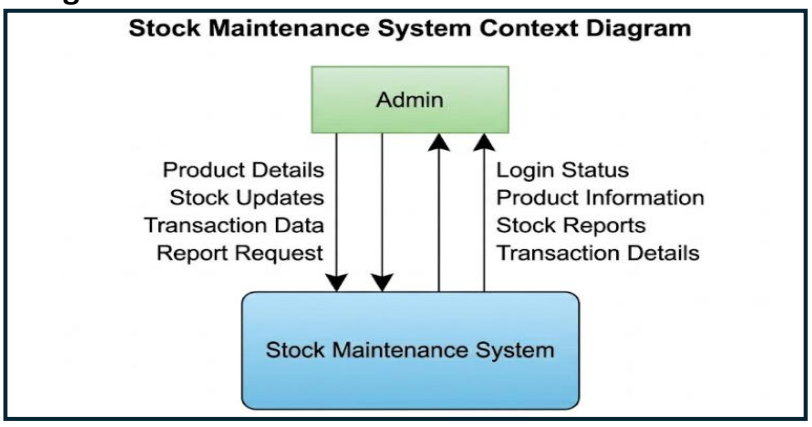
Inputs

- Login Credentials
- Product Details
- Stock Updates
- Transaction Details
- Report Requests

Outputs

- Login Status
- Product Information
- Stock Status
- Transaction Details
- Reports

Context Diagram



2. Level 0 DFD

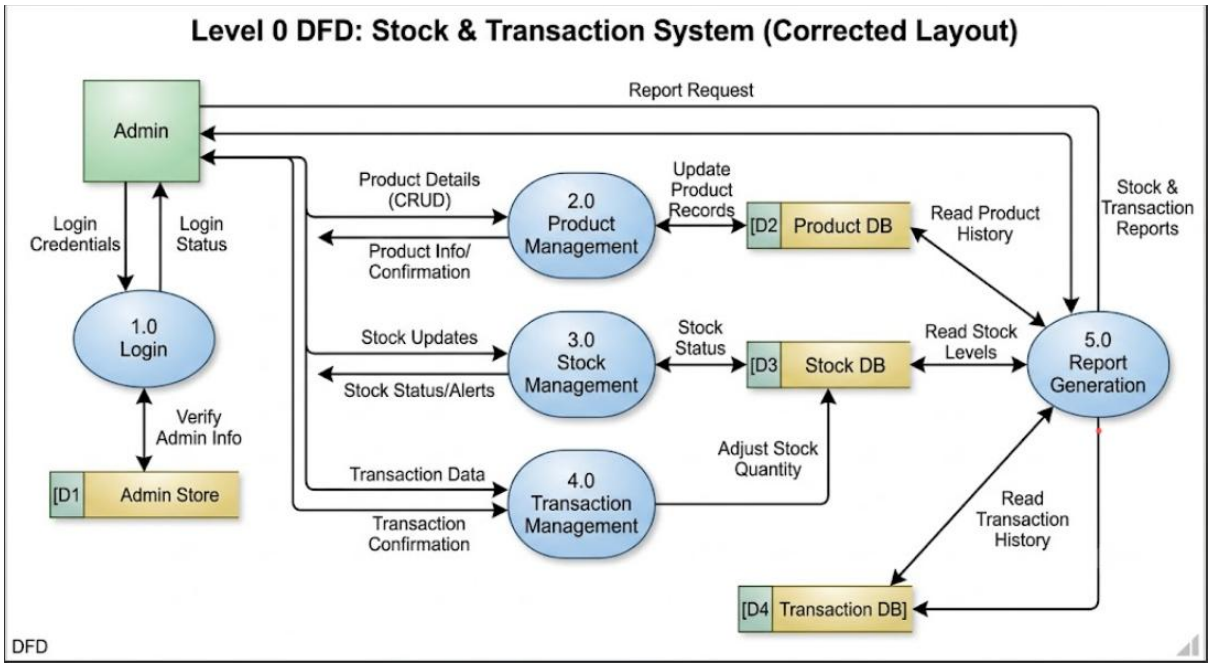
Main Processes

1. Login Management
2. Product Management
3. Stock Management
4. Transaction Management
5. Report Generation

Data Stores

- D1: Admin DB
- D2: Product DB
- D3: Stock DB
- D4: Transaction DB

Level 0 DFD



3. Level 1 DFD

Product Management

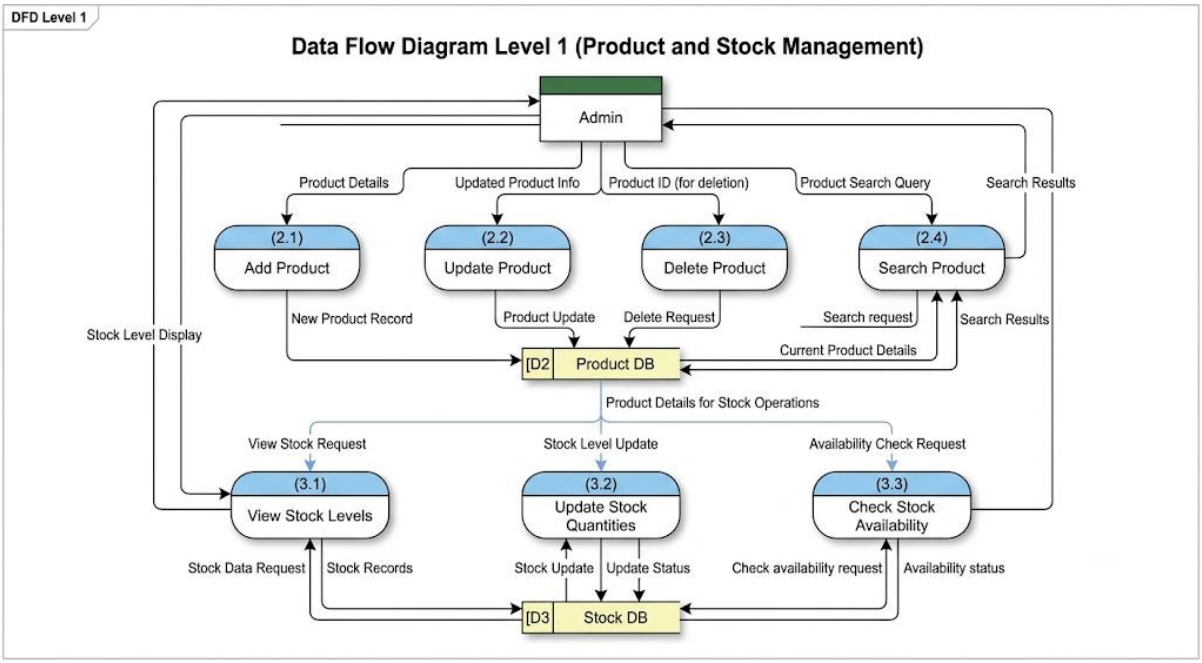
- 2.1 Add Product
- 2.2 Update Product
- 2.3 Delete Product

- 2.4 Search Product

Stock Management

- 3.1 View Stock
- 3.2 Update Stock
- 3.3 Check Availability

Level 1 DFD



Software Used

CASE Tool: Visual Paradigm

Steps Performed

1. Create a new project.
2. Select Data Flow Diagram.
3. Draw Context Diagram.
4. Create Level 0 and Level 1 DFD.
5. Save and export diagrams.

Conclusion

The DFDs were successfully developed for the Stock Maintenance System. They clearly show data flow, system processes, and database interactions, helping in system analysis and design.

TASK – 7: Performing the Structural Design using a Design Phase CASE Tool

Aim

To perform the structural design of the Stock Maintenance System using Visual Paradigm.

Objective

To represent the static structure of the system by identifying classes, attributes, methods, and relationships.

CASE Tool Used

Visual Paradigm

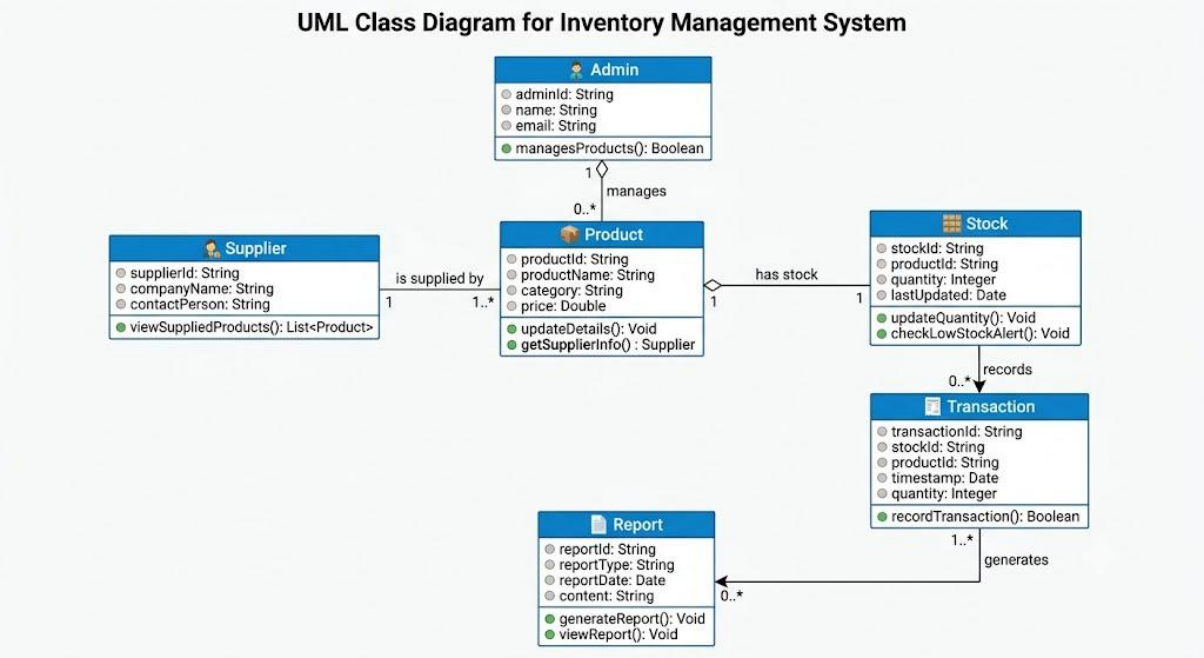
Classes Used

Class	Key Attributes	Main Methods
Admin	adminID, username, password	login(), logout()
Product	productID, productName, category, price, quantity	addProduct(), updateProduct(), deleteProduct(), searchProduct()
Supplier	supplierID, supplierName, contactNo	addSupplier()
Stock	stockID, quantity	updateStock(), viewStock()
Transaction	transactionID, date, quantity, type	recordTransaction()
Report	reportID	generateReport(), viewReport()

Relationships

- Admin manages Product.
- Supplier supplies Product.
- Product has Stock.
- Stock records Transaction.
- Transaction generates Report.

UML Class Diagram



Steps Performed

1. Open Visual Paradigm.
2. Create a new project.
3. Select Class Diagram.
4. Add classes, attributes, and methods.
5. Create relationships.
6. Save and export the diagram.

Advantages

- Shows system structure clearly.
- Identifies class relationships.
- Simplifies development and maintenance.
- Supports object-oriented design.

Conclusion

The structural design of the Stock Maintenance System was successfully created using Visual Paradigm. The UML Class Diagram provides a clear blueprint of the system's classes and relationships for implementation and maintenance.

TASK – 8: Performing the Behavioral Design using a Design Phase CASE Tool

Aim

To perform the behavioral design of the Stock Maintenance System using Visual Paradigm.

Objective

To represent the dynamic behavior of the system and show how the Admin interacts with different system functions.

CASE Tool Used

Visual Paradigm

Introduction

Behavioral design describes how the system responds to user actions. It is represented using UML diagrams such as Use Case, Activity, and Sequence Diagrams.

1. Use Case Diagram

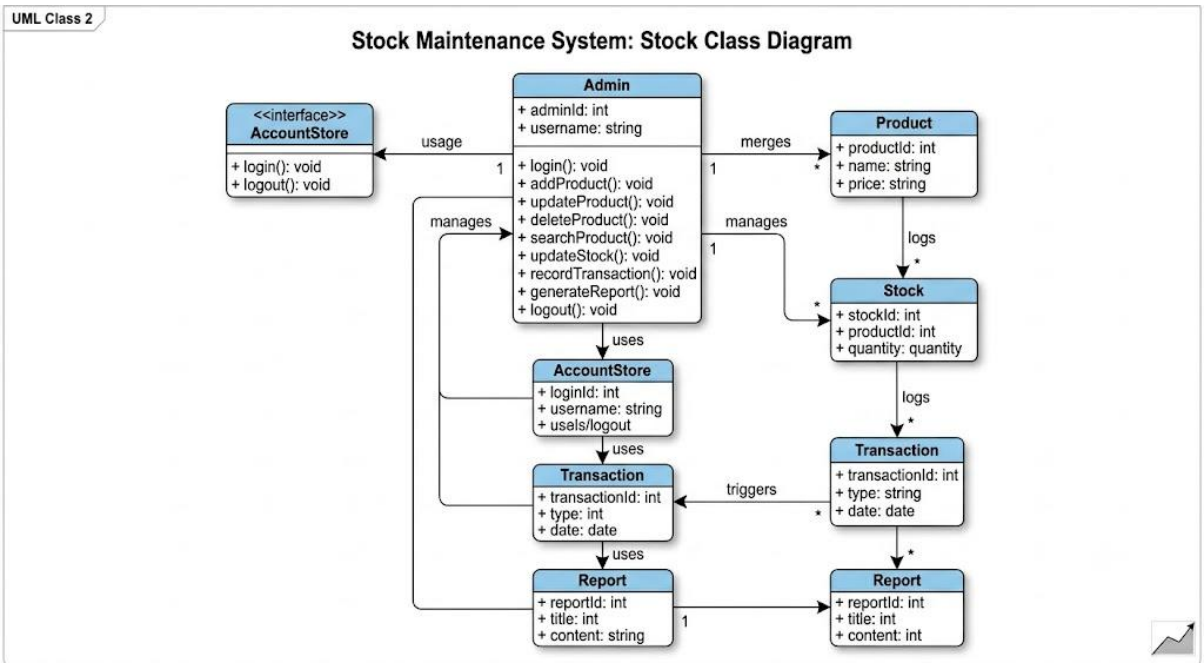
Actor

Admin

Use Cases

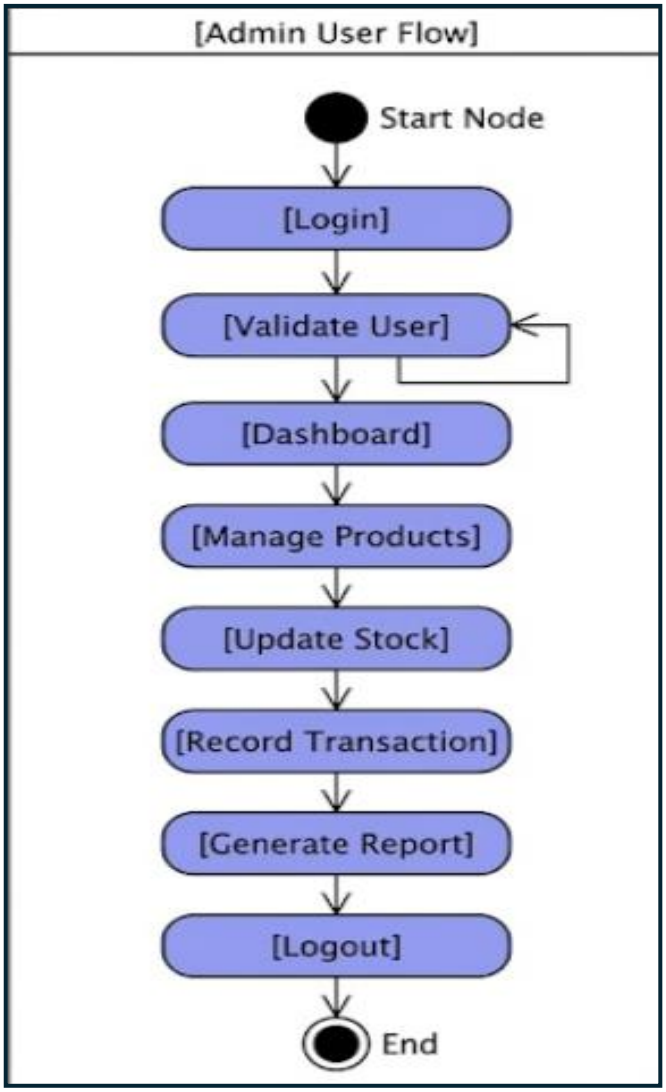
- Login
- Add Product
- Update Product
- Delete Product
- Search Product
- Update Stock
- Record Transaction
- Generate Report
- Logout

Use Case Diagram



2. Activity Diagram

Workflow



3. Sequence Diagram

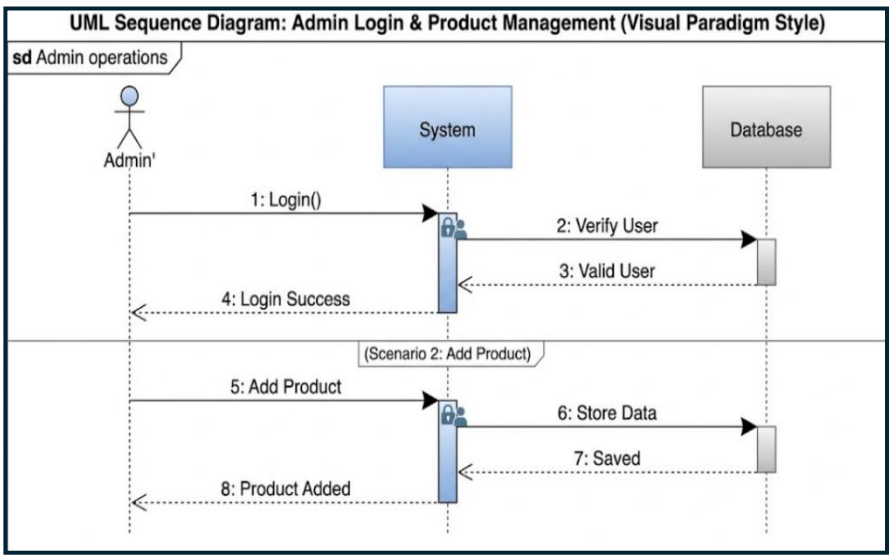
Product Management Process

Admin -> System : Login()

System -> Database : Verify User

Database -> System : Valid User

System -> Admin : Login Success



Admin -> System : Add Product

System -> Database : Store Data

Database -> System : Saved

System -> Admin : Product Added

Steps Performed

1. Open Visual Paradigm.
2. Create a new project.
3. Create Use Case Diagram.
4. Create Activity Diagram.
5. Create Sequence Diagram.
6. Save and export diagrams.

Advantages

- Shows system behavior clearly.
- Improves understanding of workflows.
- Identifies process issues early.
- Supports development and testing.

Conclusion

The behavioral design of the Stock Maintenance System was successfully developed using Visual Paradigm. The Use Case, Activity, and Sequence Diagrams provide a clear understanding of user interactions, workflows, and system operations.

TASK – 9: Study and Usage of a Software Testing Tool for Functional and Automated Testing

Aim

To study and use Selenium for performing functional and automated testing on the Stock Maintenance System.

Objective

To verify the functionality and reliability of the Stock Maintenance System through automated testing**.**

Testing Tool Used: Selenium

Introduction

Selenium is an open-source automation testing tool used for testing web applications. It automates user actions such as login, data entry, button clicks, and report generation. Selenium helps reduce manual testing effort and improves software quality.

Features of Selenium

- Supports automated testing.
- Supports multiple browsers.
- Easy integration with Java and Eclipse.
- Reduces manual testing effort.
- Supports regression testing.
- Generates reliable test results.

Software Requirements

- **Operating System:** Windows/Linux/macOS
- **IDE:** Eclipse IDE
- **Tool:** Selenium WebDriver
- **Language:** Java

Installation Procedure

1. Install Eclipse IDE.
2. Create a Java Project.
3. Download Selenium WebDriver libraries.
4. Add Selenium JAR files to the project.
5. Configure browser driver (ChromeDriver).
6. Run Selenium test scripts.

Modules Tested

1. Login Module
2. Product Management Module
3. Stock Management Module
4. Transaction Module
5. Report Generation Module

Functional Testing

Test Case	Expected Result	Status
Login Validation	Login Successful	Pass
Add Product	Product Added	Pass
Update Product	Product Updated	Pass
Delete Product	Product Deleted	Pass
Update Stock	Stock Updated	Pass
Generate Report	Report Generated	Pass

Sample Selenium Test Case

```
import org.openqa.selenium.*;

import org.openqa.selenium.chrome.ChromeDriver;
```

```
public class LoginTest {
   public static void main(String[] args) {
      WebDriver driver = new ChromeDriver();

      driver.get("http://localhost:8080");
      driver.findElement(By.id("username")).sendKeys("admin");
      driver.findElement(By.id("password")).sendKeys("admin123");
      driver.findElement(By.id("login")).click();

      System.out.println("Login Test Passed");
      driver.quit();
   }
}
```

Advantages of Selenium

- Automates repetitive testing.
- Saves time and effort.
- Supports multiple browsers.
- Improves software quality.
- Useful for regression testing.

Result

The Stock Maintenance System was successfully tested using Selenium. All modules were tested and produced the expected results.

Conclusion

Selenium was successfully used for functional and automated testing of the Stock Maintenance System. It improved testing efficiency, reduced manual effort, and ensured reliable system performance.

TASK – 10: Develop Test Cases for Unit Testing and Integration Testing

Aim

To develop test cases for Unit Testing and Integration Testing of the Stock Maintenance System.

Objective

To verify that individual modules and integrated components work correctly and meet system requirements.

1. Unit Testing

Introduction

Unit Testing checks individual modules independently to ensure proper functionality.

Unit Test Cases

Test Case ID	Module	Test Scenario	Expected Result	Status
UT01	Login	Valid Login	Login Successful	Pass
UT02	Login	Invalid Login	Error Message Displayed	Pass
UT03	Product Management	Add Product	Product Added	Pass
UT04	Product Management	Update Product	Product Updated	Pass
UT05	Product Management	Delete Product	Product Deleted	Pass
UT06	Stock Management	Update Stock	Stock Updated	Pass
UT07	Search Module	Search Product	Product Details Displayed	Pass
UT08	Report Module	Generate Report	Report Generated	Pass

2. Integration Testing

Introduction

Integration Testing verifies communication and data flow between modules.

Integration Test Cases

Test Case ID	Modules Involved	Test Scenario	Expected Result	Status
IT01	Login + Product Management	Access Product Module after Login	Module Opened	Pass
IT02	Product + Stock Management	Add Product and Update Stock	Product Stored & Stock Updated	Pass
IT03	Stock + Transaction Module	Stock Out Transaction	Transaction Recorded	Pass
IT04	Transaction + Report Module	Generate Report	Updated Report Generated	Pass
IT05	Product + Stock + Report Module	Complete Inventory Process	Correct Report Generated	Pass

Testing Summary

Testing Type	Test Cases	Result
Unit Testing	8	All Passed
Integration Testing	5	All Passed

Result

Unit Testing and Integration Testing were successfully performed. All test cases passed and produced the expected results.

Conclusion

The Stock Maintenance System was successfully tested using Unit and Integration Testing techniques. All modules and their interactions functioned correctly and satisfied the specified requirements.

TASK – 11: Develop Test Cases for White Box and Black Box Testing

Aim

To develop test cases using White Box and Black Box Testing techniques for the Stock Maintenance System.

Objective

To verify the functionality, reliability, and correctness of the system using different testing methods.

1. Black Box Testing

Introduction

Black Box Testing verifies system functionality based on inputs and outputs without examining the internal code.

Test Cases

Test Case ID	Technique	Test Scenario	Expected Result	Status
BB01	Equivalence Partitioning	Valid Login	Login Successful	Pass
BB02	Equivalence Partitioning	Invalid Login	Error Message Displayed	Pass
BB03	Boundary Value Analysis	Quantity = 0	Product Added	Pass
BB04	Boundary Value Analysis	Quantity = 1	Product Added	Pass
BB05	Boundary Value Analysis	Quantity = 9999	Product Added	Pass
BB06	Decision Table Testing	Login Validation	Correct Result Displayed	Pass

2. White Box Testing

Introduction

White Box Testing verifies internal logic, conditions, and execution paths of the program.

Test Cases

Test Case ID	Technique	Test Scenario	Expected Result	Status
WB01	Statement Coverage	Valid Login	Login Success	Pass
WB02	Statement Coverage	Invalid Login	Login Failed	Pass
WB03	Branch Coverage	Product Exists	Product Updated	Pass
WB04	Branch Coverage	Product Not Found	Error Message	Pass
WB05	Path Coverage	Search Existing Product	Product Details Displayed	Pass
WB06	Path Coverage	Search Non-Existing Product	Product Not Found	Pass

Testing Summary

Black Box Testing

Technique	Test Cases
Equivalence Partitioning	BB01, BB02
Boundary Value Analysis	BB03, BB04, BB05
Decision Table Testing	BB06

White Box Testing

Technique	Test Cases
Statement Coverage	WB01, WB02
Branch Coverage	WB03, WB04
Path Coverage	WB05, WB06

Result

All White Box and Black Box test cases were executed successfully and produced the expected results.

Conclusion

The Stock Maintenance System was tested using Equivalence Partitioning, Boundary Value Analysis, Decision Table Testing, Statement Coverage, Branch Coverage, and Path Coverage techniques. The results confirmed that the system functions correctly and meets the specified requirements.

Mini Software Engineering Case Study

AI-BASED CHATBOT SYSTEM

Name : N. Madwika Lakshmi

Roll Number : 25RBAIM0064

Branch : CSE(AI/ML)

Section : B

Subject : Software crafting and Sculpting

1. INTRODUCTION

Artificial Intelligence is transforming the way humans interact with technology. One of the most widely used AI applications is the AI-Based Chatbot System. A chatbot is a software application that can communicate with users through text or voice conversations. It uses technologies such as Artificial Intelligence (AI), Machine Learning (ML), and Natural Language Processing (NLP) to understand user queries and provide suitable responses.

Nowadays, organizations use chatbots in:

- Customer support
- Banking systems
- E-commerce websites
- Educational platforms
- Healthcare systems
- Online booking systems

Traditional customer support systems require human operators, which increases operational costs and response time. An AI chatbot reduces manual work by automating conversations and providing instant support.

The proposed AI-Based Chatbot System is designed to provide intelligent communication between users and the system. The chatbot can answer frequently asked questions, assist users, and maintain chat history.

2. PROBLEM STATEMENT

Many organizations face problems in managing customer queries manually. Human customer support systems suffer from:

- Long waiting times
- High maintenance costs
- Limited availability
- Repetitive work for employees
- Delayed responses

Customers expect instant replies and 24/7 support. Traditional systems cannot efficiently handle large numbers of users simultaneously.

Therefore, there is a need for an AI-powered chatbot system that can:

- Provide quick responses
- Reduce workload
- Improve customer satisfaction
- Automate repetitive tasks
- Work continuously without human intervention

3. OBJECTIVES

The main objectives of the AI-Based Chatbot System are:

1. To provide automated customer interaction.
2. To reduce human effort in handling repetitive queries.
3. To provide instant responses to users.
4. To maintain chat history and records.
5. To improve communication efficiency.
6. To provide 24/7 support services.
7. To increase user satisfaction through intelligent conversations.
8. To develop a scalable and reliable chatbot platform.

4. SCOPE OF THE PROJECT

The scope of the project includes:

- User login and registration
- AI-based query processing
- Chat interface
- NLP-based response generation
- Chat history maintenance
- Admin monitoring system
- Feedback collection

The system can be implemented in:

- Educational institutions
- Online shopping websites
- Banking applications
- Customer service platforms
- Healthcare support systems

5. REQUIREMENTS ENGINEERING

Requirements Engineering is the process of identifying, analyzing, and documenting user and system requirements.

A) Functional Requirements

Functional requirements describe what the system should do.

Functional Requirements of AI Chatbot System

1. User Registration
2. User Login Authentication
3. Chat Interface
4. AI Query Processing
5. Response Generation
6. Chat History Storage
7. Admin Dashboard
8. Feedback Management
9. User Management
10. Database Connectivity

B) Non-Functional Requirements

Non-functional requirements describe system quality attributes.

Non-Functional Requirements

Requirement	Description
Security	Protects user data
Reliability	System works continuously

Performance	Fast response time
Scalability	Handles many users
Maintainability	Easy software updates
Availability	24/7 service availability
Portability	Can run on multiple devices

6. SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

A) User Requirements

1. Users should register and login securely.
2. Users should communicate with chatbot easily.
3. Users should receive quick responses.
4. Users should access chat history.
5. Admin should monitor chatbot activity.

B) System Requirements

Hardware Requirements

- Intel i5 Processor or higher
- 8GB RAM
- 100GB Hard Disk
- Internet Connection

Software Requirements

- Operating System: Windows/Linux
- Frontend: HTML, CSS, JavaScript
- Backend: Python / Node.js
- Database: MySQL
- AI Library: TensorFlow / NLTK
- IDE: VS Code

C) Interface Requirements

User Interface

- Chat window
- Login page
- Dashboard

Hardware Interface

- Keyboard
- Mouse
- Mobile devices

Software Interface

- Database connectivity
- API integration

7. SYSTEM MODELING

A) USE CASE DIAGRAM

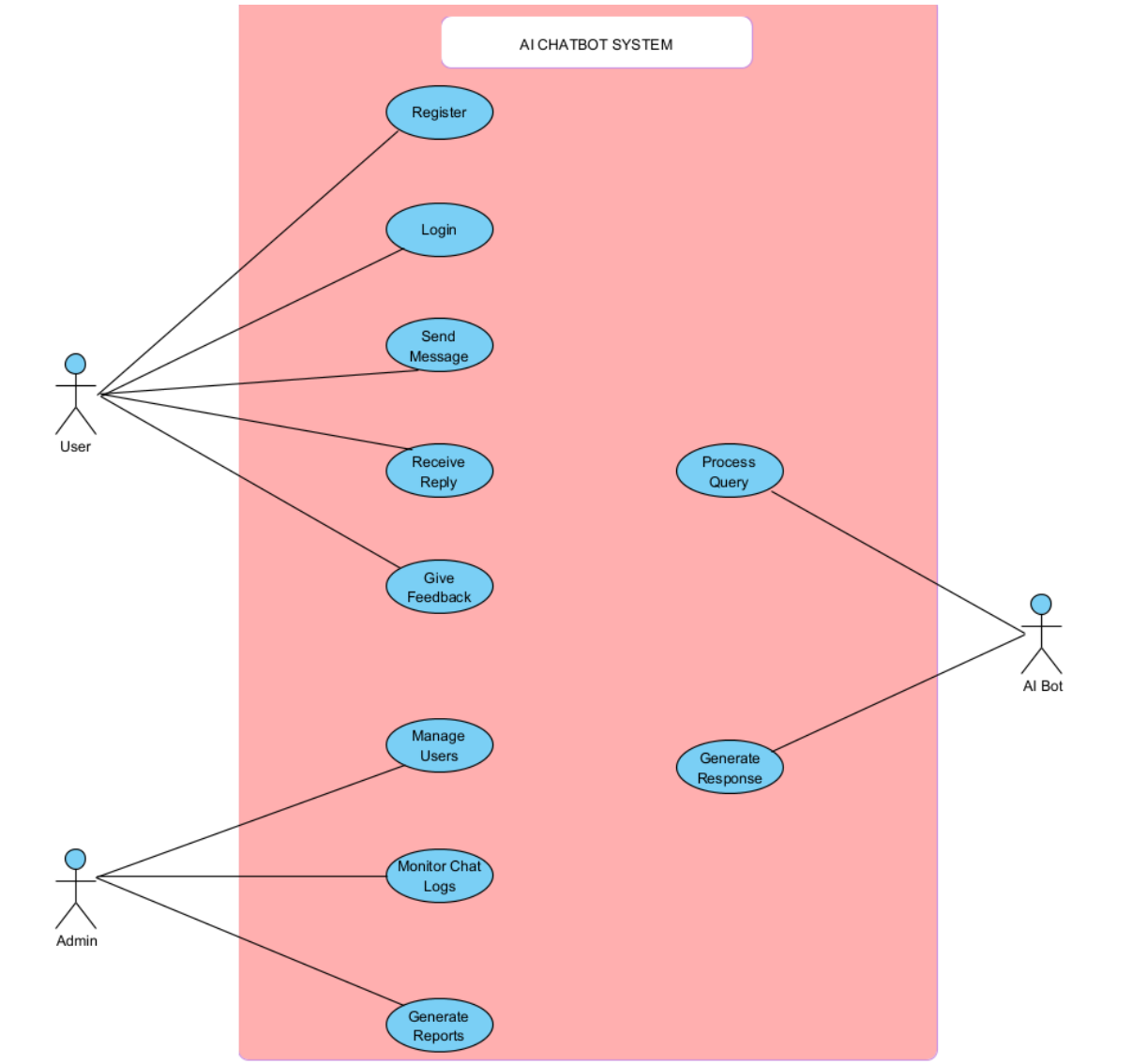
Actors

1. User
2. Admin
3. AI Chatbot

Use Cases

- Login
- Register
- Send Message
- Receive Reply
- Store Chat History
- Manage Users
- Monitor Chat Logs

Use Case Diagram



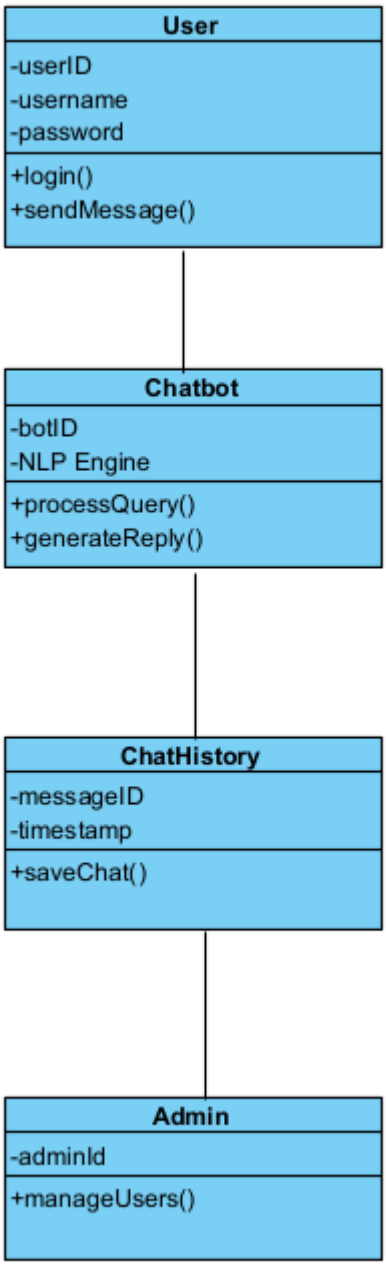
Explanation

- Users communicate with chatbot.
- Chatbot processes user queries.
- Admin manages users and monitors activities.
- AI engine generates intelligent responses.

B) CLASS DIAGRAM

- User interacts with chatbot.
- Chatbot processes queries using NLP.

- ChatHistory stores conversation data.
- Admin monitors system operations.

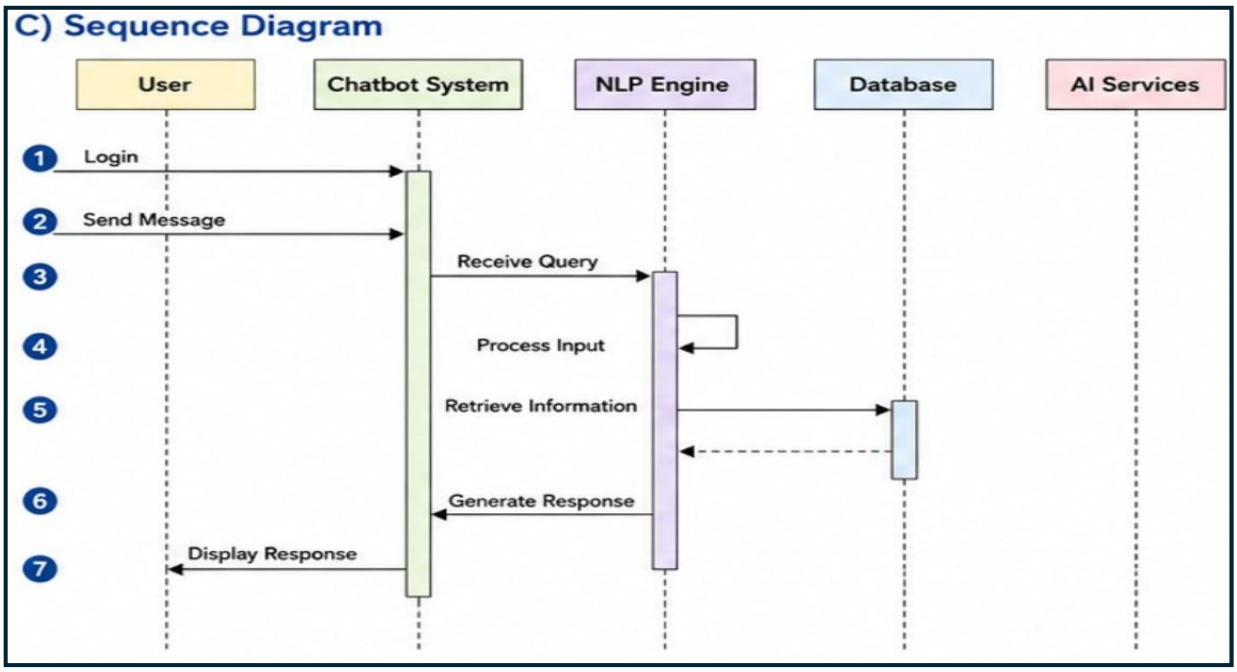


C) SEQUENCE DIAGRAM

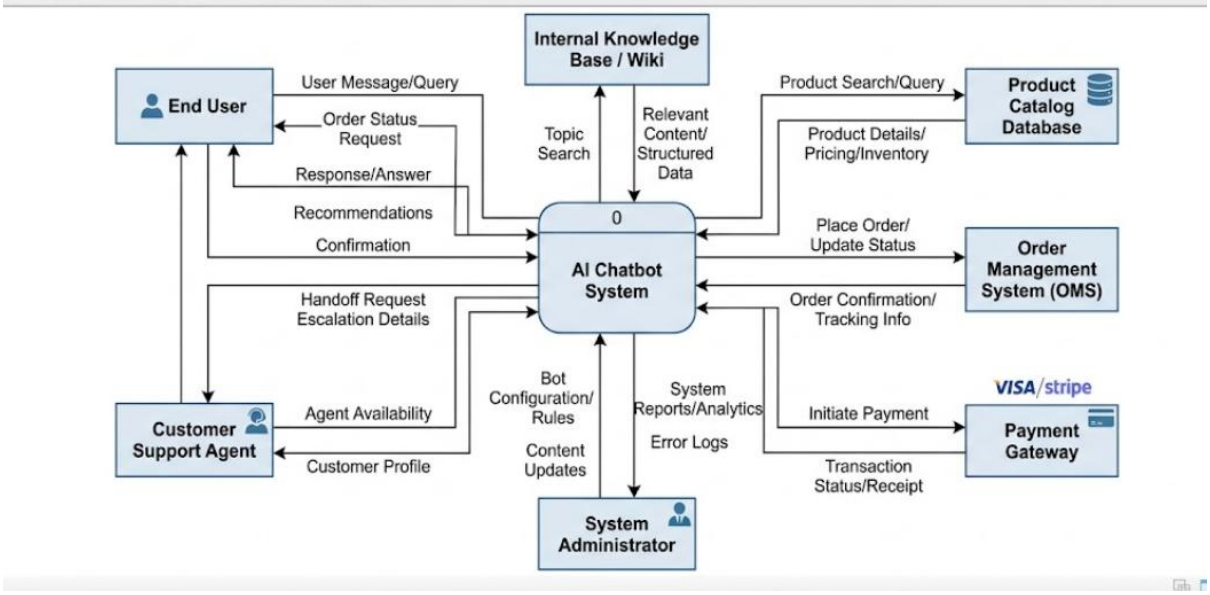
Explanation

1. User logs into system.
2. User sends message to chatbot.
3. NLP engine processes query.

4. Database retrieves information.
5. Chatbot generates response.



D) CONTEXT MODEL



Explanation

The context model shows external entities interacting with the chatbot system such as users, admin panel, and database.

E) DATA MODEL

Main Entities

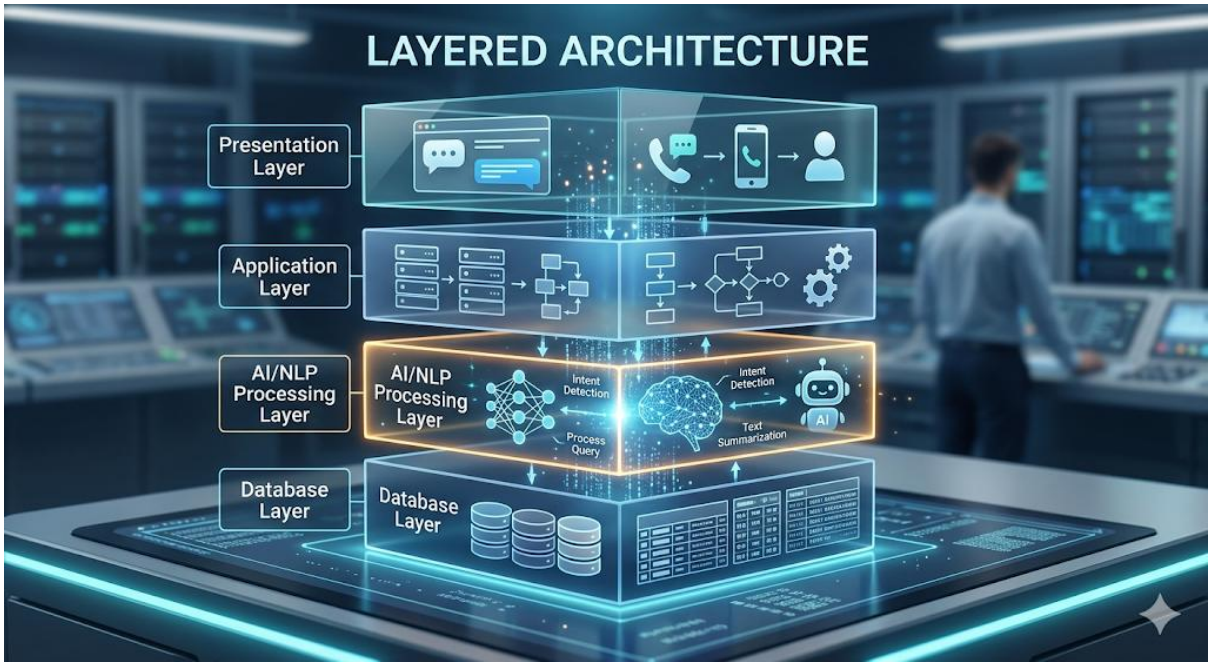
1. User
2. Admin
3. Chatbot
4. Chat History
5. Feedback
6. Responses

8. DESIGN ENGINEERING

SOFTWARE ARCHITECTURE

Layered Architecture

```
+-------------------------+
| Presentation Layer      |
+-------------------------+
| Application Layer       |
+-------------------------+
| AI/NLP Processing Layer |
+-------------------------+
| Database Layer          |
+-------------------------+
```



Explanation of Layers

1. Presentation Layer

Handles user interface and interactions.

2. Application Layer

Processes chatbot logic and request handling.

3. AI/NLP Layer

Processes natural language queries.

4. Database Layer

Stores user and chat information.

ARCHITECTURAL PATTERN

MVC Architecture

Model

Handles database operations.

View

Displays chatbot interface.

Controller

Controls user requests and chatbot responses.

DESIGN QUALITY ATTRIBUTES

Attribute	Description
Scalability	Handles large users
Security	Protects data
Reliability	Continuous operation
Maintainability	Easy updates
Performance	Faster responses
Flexibility	Easy feature addition

9. PROCESS AND RISK MANAGEMENT

PROCESS MODEL

Agile Model

Reasons for Selecting Agile

1. Continuous development
2. Faster testing
3. Frequent updates
4. Easy modification
5. Better customer feedback

RISK MANAGEMENT

RMMM PLAN

Risk	Probability	Impact	Mitigation
Server Failure	Medium	High	Backup servers
Data Leakage	Medium	High	Data encryption
Wrong AI Response	High	Medium	AI training
Heavy Traffic	Medium	Medium	Load balancing
System Crash	Low	High	Recovery system

10. TESTING STRATEGY

TEST PLAN

Test ID	Module	Expected Result
T1	Login	Successful login
T2	Chat Module	Correct response
T3	Database	Data stored
T4	Feedback	Feedback saved

BLACK BOX TESTING

Input	Expected Output
Hello	Greeting response
Invalid Password	Error message
FAQ Question	Correct answer

WHITE BOX TESTING

White box testing checks internal code logic.

Types

1. Loop Testing
2. Path Testing
3. Condition Testing

VALIDATION TESTING

Validation testing checks whether:

- User requirements are satisfied
- System behaves correctly
- Responses are accurate

SYSTEM TESTING

System testing checks the complete integrated chatbot system including:

- Frontend
- Backend
- Database
- AI module

11. SOFTWARE METRICS AND QUALITY

SOFTWARE METRICS

Metric	Description
Response Time	Time taken to reply
Defect Density	Number of defects
Test Coverage	Percentage of tested code
Accuracy Rate	Correct chatbot replies

QUALITY ASSURANCE ACTIVITIES

1. Requirement Review
2. Code Review

3. Unit Testing
4. Integration Testing
5. Documentation Review
6. Performance Testing

REVIEW TECHNIQUES

1. Peer Review

Team members review code.

2. Walkthrough

Step-by-step project explanation.

3. Inspection

Formal software checking process.

12. ADVANTAGES OF AI CHATBOT SYSTEM

1. 24/7 Availability
2. Faster Responses
3. Reduced Human Workload
4. Cost Effective
5. Better Customer Support
6. Handles Multiple Users
7. Improves Productivity
8. Consistent Communication

13. LIMITATIONS

1. Cannot handle complex emotions
2. Requires internet connection
3. AI training needed
4. Limited understanding sometimes

14. FUTURE ENHANCEMENTS

1. Voice-enabled chatbot
2. Multi-language support
3. Emotion detection
4. Video chatbot assistant
5. AI recommendation system
6. Integration with IoT devices

15. CONCLUSION

The AI-Based Chatbot System is an intelligent solution for automating customer interaction and improving communication efficiency. The system reduces manual effort, provides instant responses, and enhances user satisfaction through AI-powered conversation handling.

By implementing NLP and AI technologies, the chatbot becomes capable of understanding user queries and generating meaningful responses. The system is scalable, reliable, and suitable for modern applications such as healthcare, education, banking, and e-commerce.

The proposed system demonstrates how software engineering principles such as requirements engineering, system modeling, design engineering, testing.

AI Chatbot System

A Smart Conversational Assistant for Automated User Support

Name : N. Madwika Lakshmi
Roll No : 25RBAIM0064
Class : B. Tech (CSE(AI/ML))
Subject : Software Crafting and Sculpting

What Is an AI Chatbot?

AnAI chatbotis a software applicationthat interacts withusers through natural language —understanding intent, answering questions, and automating repetitive tasks without human intervention.

Customer Support
Instant answers to common queries

Education
24/7 student assistance

Healthcare
Triage and appointment booking

E-Commerce
Product recommendations & orders

Project Objectives

Thissystemisdesignedtobridgethegapbetween user needs and support capacity through intelligent automation.

1

Natural Language Understanding

Interpret user queries in everyday language

2

Automated Responses

Deliver relevant, accurate answers instantly

3

Reduce Manual Workload

Free human agents for complex issues

4

24/7 Operation

Always-on availability, no downtime

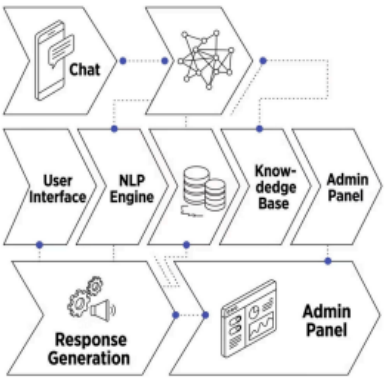
5

Improved UX

Clean, intuitive conversational interface

System Architecture

The chatbot is built from five interconnected components, each playing a distinct role in processing and responding to user input. The architecture ensures a seamless flow from user interaction to intelligent response delivery.



User Interface
Web / mobile / chat window

NLP Engine
Intent & entity recognition

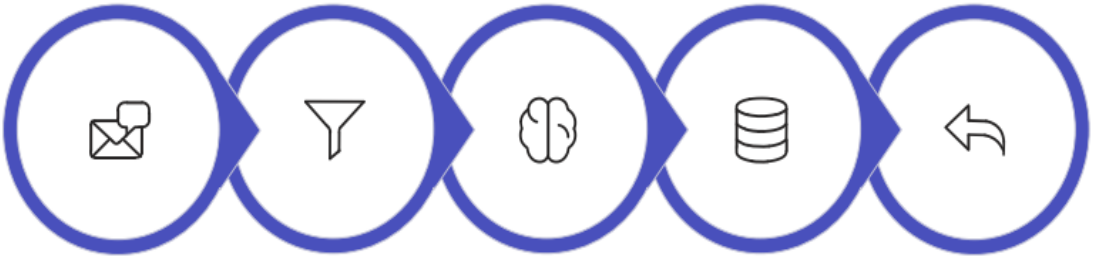
Knowledge Base
Structured data & FAQs

Response Generator
Dynamic reply construction

Admin Panel
Management & updates

How It Works

Everyconversationfollowsaclear,intelligent pipeline — from the moment a user types a message to the instant a response appears on screen.



User Message

Preprocess

Intent & Keywords

Fetch Knowledge

Generate Response

This end-to-end process typically completes in under two seconds, ensuring a smooth and responsive user experience.

Technologies Used

Py thon
Core programming language powering all backend logic

NLP Libraries
NLTK, spaCy, and Transformers for language understanding

Backend Framework
Flask / Django / FastAPI for API and routing

Database
MySQL or MongoDB for persistent data storage

Frontend
HTML, CSS, and JavaScript for the chat interface

Key Features

User Authentication

Secureloginforpersonalised sessions

Real-Time Chat Interface

Instant messaging with typing indicators

FAQ Handling

Predefinedanswers for common questions

Context-Aware Responses

Remembers conversationhistory for coherent replies

Admin Management Tools

Dashboard to update knowledge base and monitor usage

Functional Requirements

These requirements define what the system must do to meet user and business needs.

01

Accept User Queries

Receive and validate input through the chat interface

02

Process Natural Language

Tokenise, parse, and extract intent from user messages

03

Retrieve Relevant Information

Query the knowledge base for accurate, context-matched data

04

Generate & Display Responses

Construct and render replies in real time

05

Store Conversation Logs

Archive all interactions for analytics and review

06

Enable Admin Updates

Allow administrators to update the knowledge base without developer intervention

Non-Functional Requirements

Beyond features, the system must meet quality standards that define how well it performs.

Fast
Respond within seconds

Reliable
Accurate, consistent information

Secure
Protect user data and privacy

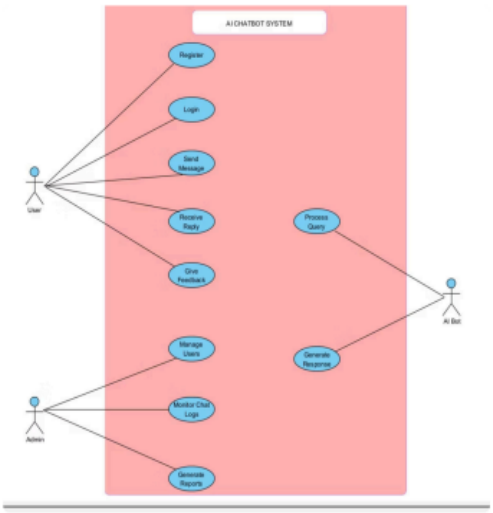
Scalable
Handle many users simultaneously

User-Friendly
Clean, intuitive interface

Summary: This AI Chatbot System delivers fast, secure, and scalable automated support — reducing costs while improving the user experience.

Use Case Diagram

The systemdefines two primaryactors— **User**and **Admin** — each with distinct interactions. Users send messages, receive AI-poweredresponses, and viewchathistory. Admins manage the knowledge base and access usage reports, ensuring the system stays accurate and up to date.



User Actions

- Send Message
- Receive Response
- View Chat History

Admin Actions

- Manage Knowledge Base
- View Reports

Testing Strategy

A structured four-layertestingapproachensures the chatbot performs reliably across all scenarios — from individual components to end-user experience.

Unit Testing

Validate individual components in isolation

Integration Testing

Verify modules communicate correctly

System Testing

Test the full system end-to-end

UAT

Confirm readiness with real users

Test Case	Expected Result
Valid query	Correct response displayed
Unknown query	Fallback message displayed
Database connection	Successful data retrieval

Advantages & Limitations

Advantages

- **24/7 Availability** — Always on, no downtime
- **Quick Response Time** — Instant replies to user queries
- **Reduced Operational Cost** — Automates routine support tasks
- **Consistent Answers** — Standardized responses every time

Limitations

- **Complex Queries** — May misinterpret nuanced or ambiguous input
- **Training Data Dependency** — Quality of responses depends on data quality
- **Regular Updates Required** — Knowledge base needs ongoing maintenance

Future Enhancements

Thesystemisdesignedtoevolve.Theseplannedenhancements will extend its capabilities and improve the overall user experience.

Voice-Based Interaction

Enable spoken queries for hands-free accessibility

Multilingual Support

Respond in multiple languages to serve a wider audience

Sentiment Analysis

Detect user mood and adapt responses accordingly

External API Integration

Connect to third-party services for richer data

Personalized Recommendations

Tailor responses based on user history and preferences

Conclusion

The **AI Chatbot System**automates communication and support using natural language processing —improving efficiency, reducing workload, and delivering quick, reliable assistance to users around the clock.

Automates

Routine queries and support tasks

Improves

Operational efficiency and response speed

Sc ales

With advanced AI capabilities for future growth

With planned enhancements — voice interaction, multilingual support, and sentiment analysis — the system is well-positioned to deliver an even more intelligent and personalized user experience.